Title: Estimation of Distribution Algorithms

Name: Pedro Larrañaga†, Concha Bielza

Affil./Addr.: Departamento de Inteligencia Artificial, Universidad Politécnica de Madrid, Madrid, Spain

e-mail: pedro.larranaga@fi.upm.es; mcbielza@fi.upm.es

†Corresponding author

Estimation of Distribution Algorithms

Abstract

In this chapter estimation of distribution algorithms will be described. These algorithms belong to the evolutionary computation field and are characterized by evolving a population of candidate individuals as solutions of the optimization problem estimating at each generation the joint probability distribution of the selected individuals and then sampling new individuals from that distribution. Thus how to model (and sample from) that joint distribution is an important issue that in this chapter will be analysed. The specificities of these algorithms when dealing with multimodal, multiobjective, or dynamic optimization problems will also be overviewed. Then parallelization and hybridations with other optimization heuristics will be presented. Next, applications of the algorithms will come in the last two sections. On the one hand, the use of these algorithms to solve real-world optimization problems in biomedicine, bioinformatics, energy, vehicle routing and scheduling will be shown. On the other hand, applications in different machine learning tasks, such as supervised classification, clustering and Bayesian networks will be discussed. Finally, the conclusions will round the chapter off.

Keywords

Estimation of distribution algorithms. Joint probability distribution. Sampling from a distribution. Bayesian network. Probabilistic dependence. Applications in optimization. Applications in machine learning.

Introduction

Estimation of distribution algorithms (EDAs) [1–6] are a class of evolutionary algorithms that build probabilistic models of promising solutions to guide the search process in optimization. Unlike traditional genetic algorithms, EDAs do not use crossover and mutation but instead learn and then sample from a probability distribution that represents the best solutions found so far, gradually refining this distribution to converge to optimal or near-optimal solutions.

This method of evolving populations of candidate individuals toward the optimum of a function offers several advantages. Firstly, the evolutionary process is stochastic and grounded in robust probability theory, making easier to escape from local optima. Secondly, there is no need to design *ad hoc* operators for each specific problem, as is often required in other evolutionary computation methods. Thirdly, interactions among the variables encoding each individual, represented through conditional (in)dependence, can be explicitly observed if a probabilistic graphical model is learned from the selected population at each generation. This feature enhances the interpretability of the search process. Fourthly, it is possible to incorporate domain expertise from the optimization process by embedding expert knowledge directly into the graphical model, such as by enforcing certain must-link or must-not-link constraints between variables within the graph.

An Illustrative Example

To enhance comprehension of the distinct components and stages of EDAs, the most straightforward version of this method is applied to a basic optimization scenario. Consider the objective of maximizing the LeadingOnes function, that is, a function which maps a bit string of length n to its number of 1s before the first 0 in the bit string, wherein the goal is to achieve the function's maximum output [7]. Formally, for $\mathbf{x} = (x_1, ..., x_n)$, $\textsc{LeadingOnes}(\mathbf{x}) = \sum_{i=1}^{n} \prod_{j=1}^{i} x_j$. Its maximum is achieved at point $\mathbf{x}^* = (1, ..., 1)$ and its function value is n.

Consider a 6-dimensional space, that is, $n = 6$. The initial population can be generated randomly. Let D_0 denote the dataset containing 20 instances (see Table 1) resulting from this generation process.

In a subsequent phase, the most fitted individuals are selected from D_0. In this case truncation selection is used, whereby half of the population is retained. Let D_0^{Se} denote the dataset comprising the selected individuals, where in this case 9 out of the 20 individuals have fitness values greater than 0 and for the rest this value is 0. Thus, from the 11 individuals of fitness 0 one individual is selected, which is usually done at random. Following the selection of these 10 individuals (see Table 2) their characteristics are aimed to be captured through a joint probability distribution. While it should ideally account for all dependencies among the variables, in this example, the simplest model to represent the joint probability distribution is employed, where each variable is considered independent of the others. Mathematically, this assumption is formulated as follows: $p_1(\mathbf{x}) = p_1(x_1, \ldots, x_6) = \prod_{i=1}^{6} p_1(x_i \mid D_0^{Se})$.

Thus, the model specification requires only six parameters. Each parameter, $p(x_i \mid D_0^{Se})$ for $i = 1, \ldots, 6$, is estimated from D_0^{Se} based on the corresponding relative frequency:

Table 1. The initial population D_0. Each individual is evaluated with the LeadingOnes function

Individual	X_1	X_2	X_3	X_4	X_5	X_6	$f(\mathbf{x})$
1	0	0	1	0	1	0	0
2	0	1	0	0	1	0	0
3	0	0	0	1	0	0	0
4	1	1	1	0	0	1	3
5	0	0	0	0	0	1	0
6	1	1	0	0	1	1	2
7	0	1	1	1	1	1	0
8	0	0	0	1	0	0	0
9	1	1	0	1	0	0	2
10	1	0	1	0	0	0	1
11	1	0	0	1	1	1	1
12	1	1	0	0	0	1	2
13	1	0	1	0	0	0	1
14	0	0	0	0	1	1	0
15	0	1	1	1	1	1	0
16	0	0	0	1	0	0	0
17	1	1	1	1	1	0	5
18	0	1	0	1	1	0	0
19	1	0	1	1	1	1	1
20	0	0	1	1	0	0	0

$$\hat{p}_1(X_1 = 1 \mid D_0^{Se}) = 0.9 \quad \hat{p}_1(X_2 = 1 \mid D_0^{Se}) = 0.5 \quad \hat{p}_1(X_3 = 1 \mid D_0^{Se}) = 0.6$$

$$\hat{p}_1(X_4 = 1 \mid D_0^{Se}) = 0.4 \quad \hat{p}_1(X_5 = 1 \mid D_0^{Se}) = 0.5 \quad \hat{p}_1(X_6 = 1 \mid D_0^{Se}) = 0.5$$

By sampling from this joint probability distribution, $p_1(\mathbf{x})$, a new population D_1 of 20 individuals is generated. Table 3 presents these individuals evaluated according to $f(\mathbf{x})$. Subsequently, the top 10 individuals are selected from D_1, resulting in the dataset D_1^{Se}, as shown in Table 4. From D_1^{Se}, the estimations are:

Table 2. The selected individuals D_0^{Se}, from the initial population

Individual	X_1	X_2	X_3	X_4	X_5	X_6
1	0	0	1	0	1	0
4	1	1	1	0	0	1
6	1	1	0	0	1	1
9	1	1	0	1	0	0
10	1	0	1	0	0	0
11	1	0	0	1	1	1
12	1	1	0	0	0	1
13	1	0	1	0	0	0
17	1	1	1	1	1	0
19	1	0	1	1	1	1

$$\hat{p}_2(X_1 = 1 \mid D_1^{Se}) = 1 \qquad \hat{p}_2(X_2 = 1 \mid D_1^{Se}) = 1 \qquad \hat{p}_2(X_3 = 1 \mid D_1^{Se}) = 1$$

$$\hat{p}_2(X_4 = 1 \mid D_1^{Se}) = 0.6 \quad \hat{p}_2(X_5 = 1 \mid D_1^{Se}) = 0.6 \quad \hat{p}_2(X_6 = 1 \mid D_1^{Se}) = 0.5$$

With these first two generations of the EDA, observe that the mean of $f(x)$ in D_0^{Se} was 1.9, while it has now increased to 4.1 in D_1^{Se}.

These three steps: (i) selecting individuals from the population, (ii) learning the joint probability distribution of the selected individuals, and (iii) sampling from the learned distribution, are repeated until a predetermined stopping criterion is met.

The primary challenge with EDAs lies in the estimation of the probability distribution $p_l(\mathbf{x})$ at each generation l. Clearly, estimating all parameters necessary to specify this joint probability distribution is impractical. Consequently, various approximations have been proposed, wherein the joint probability distribution is assumed to factorize according to a chosen probabilistic model.

Table 3. The population of the first generation D_1. Each individual is evaluated with the LeadingOnes function

Individual	X_1	X_2	X_3	X_4	X_5	X_6	$f(\mathbf{x})$
1	1	1	1	1	1	0	5
2	1	1	1	0	0	1	3
3	1	0	0	0	1	1	1
4	1	1	1	1	0	0	4
5	1	0	0	0	0	0	1
6	1	1	1	0	1	1	3
7	1	1	1	0	0	1	3
8	1	0	0	1	1	0	1
9	1	1	1	1	0	0	4
10	1	0	1	0	0	0	1
11	1	0	0	1	1	1	1
12	1	1	0	0	0	1	2
13	1	0	1	0	0	0	1
14	1	0	0	0	1	1	1
15	1	1	1	0	1	1	3
16	1	0	0	0	0	0	1
17	1	1	1	1	1	0	5
18	1	1	1	1	1	0	5
19	1	1	1	1	1	1	6
20	1	0	0	0	0	1	1

General Scheme of EDAs

The probability distribution that governs the individuals selected in each generation based on their fitness is then sampled to generate a new population of candidate solutions. For a comprehensive recent review, see [8].

Table 4. The selected individuals D_1^{Se}, from the first generation D_1

Individual	X_1	X_2	X_3	X_4	X_5	X_6
1	1	1	1	1	1	0
2	1	1	1	0	0	1
4	1	1	1	1	0	0
6	1	1	1	0	1	1
7	1	1	1	0	0	1
9	1	1	1	1	0	0
15	1	1	1	0	1	1
17	1	1	1	1	1	0
18	1	1	1	1	1	0
19	1	1	1	1	1	1

Algorithm 1 presents a general and unified pseudocode that encompasses all variants of EDAs introduced below. The process begins by generating an initial population of individuals.

Algorithm 1: Pseudocode for the EDA

$D_0 \leftarrow$ Generate M individuals (initial population)

repeat for $l = 1, 2, ...$

1. $D_{l-1}^{Se} \leftarrow$ Select $N \leq M$ individuals from D_{l-1} according to the selection method

2. $p_l(\mathbf{x}) = p(\mathbf{x}|D_{l-1}^{Se}) \leftarrow$ Estimate the probability distribution of an individual $\mathbf{x}$ being among the selected population

3. $D_l \leftarrow$ Sample M individuals (new population) from $p_l(\mathbf{x})$

until *the stopping criterion is met*

In the first phase, the M generated individuals are evaluated, and a subset of N individuals is chosen based on a predetermined selection criterion. This criterion can be deterministic (e.g., selecting the N individuals with the highest objective function

values) or stochastic, where individuals are selected with a probability proportional to their evaluation.

The second phase involves estimating the joint probability distribution of the selected individuals under various assumptions on the interactions among the decision variables encoding each individual. In the EDA literature, three common scenarios are considered: (a) assuming that all variables are independent, (b) accounting for only bivariate dependencies, or (c) allowing for unrestricted dependencies among the variables. In most practical optimization problems, the independence assumption in (a) is unrealistic, whereas scenario (c) can be computationally prohibitive when dealing with a high number of variables.

The third phase uses the estimated probability distribution to sample a new population of individuals. In cases of constrained optimization problems, it is essential to ensure that the generated individuals satisfy the constraints during the sampling process.

These three steps —evaluation and selection of individuals, learning of the probabilistic model, and simulation— are repeated until a stopping criterion is met. The stopping condition could be based on the number of generations, on the convergence towards a global optimum, or on a predefined limit on execution time.

Initializing the Population

The simplest way to initialize the population is by sampling from a uniform distribution across each variable, thereby obtaining individuals that cover the entire search space in a balanced manner. If prior knowledge about the problem is available or even historical data, the initialization process could use them.

Selection of Individuals

The selection of individuals in each generation of an EDA can be carried out through common and straightforward procedures in evolutionary computation [9], such as elitist selection, roulette wheel selection, truncation selection, tournament selection, rank-based selection, or more sophisticated methods like Boltzmann selection or stochastic universal sampling.

Learning the Joint Probability Distribution

Discrete EDAs refer to a class of optimization algorithms grounded in the EDA framework, specifically tailored for addressing combinatorial optimization challenges. On the other hand, continuous EDAs are utilized to tackle optimization in continuous spaces. Both discrete [5] and continuous [10] EDAs are classified according to the probabilistic dependency structures allowed for $p_l(\mathbf{x})$ in Algorithm 1.

No dependencies. The n-dimensional joint probability distribution of selected individuals in each generation is assumed to decompose as a product of n univariate probability distributions, one for each variable. In other words, $p_l(\mathbf{x}) = \prod_{i=1}^{n} p_l(x_i)$, where in continuous EDAs, densities replace p_l. Key algorithms in this category include: (a) the univariate marginal distribution algorithm (UMDA) [11] for discrete problems, the univariate marginal distribution algorithm for continuous domains (UMDA$_c^G$) where each variable assumes a univariate Gaussian distribution [10]; (b) population-based incremental learning (PBIL) [12], which updates the probability vector components using a Hebbian rule inspired by the selected individuals in each generation, and its continuous version [13], where Gaussianity is assumed for each marginal univariate distribution; and (c) the compact genetic algorithm (cGA) [14], which simulates only two individuals per generation from the current probability distribution, iteratively adjusting probabilities toward the more successful individual until convergence. A general

framework for univariate discrete EDAs that encompasses UMDA, PBIL, and cGA is proposed in [15].

Bivariate dependencies. The foundational works in this domain include: (a) mutual information maximization for input clustering (MIMIC), (b) combining optimizers with mutual information trees (COMIT), and (c) the bivariate marginal distribution algorithm (BMDA). In MIMIC [16], the best variable permutation is searched for in each generation to approximate the empirical distribution of selected individuals by minimizing the Kullback-Leibler divergence, where $p_l^{\pi}(\mathbf{x}) = p_l(x_{i_1}|x_{i_2})p_l(x_{i_2}|x_{i_3}) \cdots p_l(x_{i_{n-1}}|x_{i_n})p_l(x_{i_n})$, with $\pi = (i_1, i_2, ..., i_n)$ denoting a permutation of indices. MIMIC$_c^G$ [10] extends MIMIC for continuous problems by assuming Gaussianity for both marginal and conditional distributions. In COMIT [17], a tree structure is learned using the maximum weight spanning tree algorithm in each generation. In [18] the dependency tree is based on bivariate copula functions. BMDA [19] factorizes the joint probability distribution according to an acyclic graph formed by a set of trees (forest), where dependencies between pairs of variables are considered based on a threshold. The search of the forest has also been carried out using normalized mutual information [20].

Multivariate dependencies. EDAs in this category typically focus on learning a Bayesian network (or a Gaussian Bayesian network) [21] that best represents the selected individuals' distribution in each generation, followed by sampling from the learned model. Key examples include: (a) the estimation of Bayesian network algorithm (EBNA) [22], [23], which explores the learning phase both with constraint-based and score-based approaches, with BIC and K2 being the prominent scores; EGNA [10], a variant for continuous problems that learns a Gaussian Bayesian network in each generation, and [24], which adapts EGNA for high-dimensional issues by controlling model complexity; (b) the Bayesian optimization algorithm (BOA) [25], which utilizes

the so-called BDe score and a greedy search method starting from scratch in each generation; (c) the learning factorized distribution algorithm (LFDA) [26], which limits the Bayesian network's complexity via the BIC score and a restriction on the number of parent variables per node; (d) the estimation of multivariate normal algorithm (EMNA) [10], which assumes a Gaussian joint probability distribution and estimates the mean vector and covariance matrix using maximum likelihood; (e) regularization-based EDAs, which apply regularization during Bayesian network learning [27], or which introduce regularized Gaussian Bayesian networks to address high-dimensional continuous problems [28]; and (f) the extended compact genetic algorithm (EcGA) [29], which factorizes the joint probability distribution as a product of distributions of variable groups, without the need for learning a Bayesian network every generation. Recent works on continuous optimization include the use of denoising autoencoders [30], restricted Boltzman machines using attention mechanisms [31] and semiparametric Bayesian networks [32].

Sampling of New Candidate Solutions

Sampling in EDAs is mainly based on stochastic simulation methods of Bayesian networks and Gaussian Bayesian networks. For the former, there are methods such as probabilistic logic sampling, rejection sampling, likelihood weighting, importance sampling, Gibbs sampling, Markov chain Monte Carlo or variational inference sampling, whereas for the latter, possible methods are the Cholesky decomposition, eigenvalue decomposition, principal component analysis sampling, or Box-Muller transformations.

Practical Issues

From a practical perspective EDAs have been adapted to tackle special (although common) optimization settings. In problems involving the search for the best **per-**

mutation, which naturally arises in the traveling salesman problem, vehicle routing, or scheduling, EDAs have fundamentally adopted four ways of representing individuals: (i) direct permutation representation, in which the individual directly represents a permutation; (ii) adjacency representation, where each position in the sequence indicates the next element in the permutation; (iii) position representation, where each element of the permutation is assigned a specific position in the sequence, rather than directly encoding the order of elements or their adjacency relationships; and (iv) a real-coded individual, where the sorted indices represent the order of items in the permutation. The probabilistic modeling of permutations of selected individuals in each generation (see a review in [33]) can be carried out using position-based models, where the probability distribution is defined over the position of each element in the permutation independently of the others. Alternatively, it can be done using probability distributions defined over the permutation space, such as the Mallows distribution, the generalized Mallows distribution, or the Plackett-Luce model. These special probability distributions for permutations capture dependencies across the entire permutation, making them better suited for more complex rankings and ordering scenarios, as they account for the relative positions of all elements in the permutation.

Multimodal optimization is the process of identifying multiple optimal solutions within a search space. This type of optimization is valuable for problems where diverse solutions are desired, allowing for a more comprehensive exploration of potential solutions rather than focusing solely on a single global optimum. Multimodal optimization problems can be very challenging for an EDA based on a single probability distribution. Therefore, the most common approach to such problems is to consider a finite mixture of probability distributions, each component of the mixture corresponding to one of the peaks of the objective function. Partition-based clustering is used in [34] for multimodal combinatorial optimization problems, while [35] applies it

in continuous domains. The structural expectation-maximization (EM) algorithm for multimodal problems in combinatorial optimization (based on Bayesian networks), as well as in continuous domains (based on conditional Gaussian Bayesian networks), is proposed in [36].

Multiobjective optimization is the process of simultaneously optimizing two or more conflicting objectives, aiming to find solutions that balance trade-offs between them. Rather than a single optimal solution, a set of Pareto-optimal solutions is yielded, where improving one objective would worsen at least one another. Evolutionary computation in general and EDAs in particular are well-suited for multiobjective optimization because it naturally handles complex, conflicting objectives and efficiently explores diverse solutions within a population. There is a large number of references that use EDAs in multiobjective optimization. Among the most notable is the multiobjective hierarchical BOA [37] applied to multiobjective decomposable problems, and the proposal in [38] where a multidimensional Gaussian Bayesian network is used to learn a joint model of objectives and variables while at the same time differentiating their role in the network.

Dynamic optimization involves finding optimal solutions for problems where the decision variables and/or the objective function change over time. EDAs have been used in dynamic optimization for both single-objective problems [39], with a PBIL version based on multiple populations, and multiobjective problems [40], leveraging transfer learning.

The **parallelization** of EDAs is justified by the high computational cost associated with the three steps these algorithms comprise. Particularly costly is the learning of the joint probability distribution of the selected individuals when modeled through (Gaussian) Bayesian networks. While early parallel EDAs [41] focused on parallelizing only this step, later approaches extended parallelization to all three steps of EDAs [42].

While EDAs perform well in exploring the search space, they are not as effective in its exploitation [43]. This is the main reason why, in real-world problems, EDAs are often **hybridized** with other heuristic strategies. In the literature, there are some works on EDAs hybridized with local search, particle swarm optimization, genetic algorithms, variable neighborhood search, simulated annealing, ant colony optimization, scatter search, or tabu search. However, as we have checked in a literature review (unpublished) of EDA papers published in the Web of Science until 2024, the heuristic that appears most frequently in hybridization with EDAs is differential evolution [44].

Applications in Optimization

EDAs have been applied to solving real-world problems across a wide range of domains. In **biomedicine**, they have been used to predict the survival of cirrhotic patients who have undergone a transjugular intrahepatic portosystemic shunt, which is a procedure to create new connections between two blood vessels in the liver [45]. EDAs have also been applied to the treatment of epithelial ovarian cancer, the search for the optimal chemotherapy dose in cancer, variable selection in a severity classification system for Parkinson's patients based on non-motor symptoms, and a nurse shift scheduling problem. In **bioinformatics**, EDAs have been used to solve the problem of protein structure prediction in simplified models [46], along with other problems such as protein folding, side chain placement, motif discovery, and the search for genetic regulatory networks.

The determination of the best possible locations of depleted uranium bundles, which maximize fuel economy as well as safety, is a large-sized combinatorial optimization problem with constraints that have been approached with EDAs [47] in the **energy** domain. Other problems where EDAs have proposed accurate solutions in this domain are the optimal management of a large number of plug-in hybrid electric vehicles charging at a municipal parking station, the search for the optimal model

parameter estimation of solar and fuel cells, and the prediction of the possible gas emission quantity in coal mines.

In **vehicle routing**, EDAs have provided solutions to: the school bus routing problem with stops selection, an equitable risk routing problem for shipments of hazardous materials, truck and trailer routing problems, the vehicle routing problem with time windows, an agricultural route planning problem, trajectory optimization of space vehicle in rendezvous proximity operation, and signal optimization for large-scale traffic networks [48].

An artificial immune algorithm combined with an EDA for the **traveling salesman problem** (TSP) is proposed in [49]. EDAs have also been developed for some variants of the TSP: multiobjective multiple TSP or for the travelling thief problem [50].

Scheduling is the optimization problem for which the most EDA-based approaches have been developed. This includes some variants of the job shop scheduling problem, such as the job shop scheduling with sequence-dependent, the no-wait job shop scheduling, the flexible job shop scheduling and the multi-objective flexible job shop scheduling, as well as variants of the flow shop scheduling problem, such as the blocking flow shop scheduling, the hybrid flow shop scheduling, the hybrid flow shop scheduling with identical parallel machines, the hybrid flow shop scheduling with stochastic processing time, the reentrant hybrid flow shop scheduling, and the no-idle permutation flow shop scheduling.

Applications in Machine Learning

Machine learning is currently the area of artificial intelligence that is developing the most interesting methodological advancements and practical applications. A dataset serves as the input to an algorithm capable of inducing a model to make rational decisions. The search for the model (including its structure and parameters) that best

fits the data or yields the best performance is conducted within a search space of such high dimensionality that an exhaustive search becomes unfeasible.

The exploration of high-dimensional search spaces is inherent to various machine learning tasks, as illustrated by the following examples:

- *Feature subset selection*, where the total number of possible feature subsets, denoted $f(n)$ for n features, is $f(n) = 2^n$;
- *Partitional clustering*, where the number of possible partitions of N objects into K clusters is given by $S(N, K) = \frac{1}{K!} \sum_{i=0}^{K} (-1)^{K-i} \binom{K}{i} i^N$, for $K \in \mathbb{N}$, with initial conditions $S(0, 0) = 1$ and $S(N, 0) = S(0, N) = 0$;
- *Learning a Bayesian network structure* with n nodes from data within the space of directed acyclic graphs (DAGs), where the number of possible structures is calculated using the recursive relation $f(n) = \sum_{i=1}^{n} (-1)^{i+1} \binom{n}{i} 2^{i(n-i)} f(n-i)$ for $n > 2$, with initial conditions $f(0) = f(1) = 1$;
- *Permutation-based problems*, such as optimal triangulation in the moral graph of a Bayesian network with n nodes, where the search space cardinality is $n!$.

In each of these cases, the search is conducted in a discrete space, posing a highly complex combinatorial optimization problem. Additionally, determining optimal parameters for a machine learning model generally involves continuous optimization challenges, for instance, solving the likelihood equations in logistic regression which lacks a closed-form solution, or training an artificial neural network where its parameters are optimized using the backpropagation algorithm, a process often prone to local optima.

The complexity of this situation justifies the use of metaheuristics for searching the optimal machine learning model, leading to the field known as evolutionary machine learning when these heuristics are based on evolutionary computation. This section

reviews several selected works in which EDAs are employed for this purpose. See [51] for a more complete discussion on this topic.

Supervised classification is one of the machine learning tasks where EDAs have been utilized. In feature subset selection for classifiers such as naive Bayes, the EBNA algorithm has been applied [52], as well as the UMDA$_c^G$ algorithm in logistic regression [53]. In artificial neural networks, the PBIL algorithm has been used as an alternative to backpropagation for finding optimal weights connecting nodes in consecutive layers of a multilayer perceptron [54], while the UMDA$_c^G$ has been applied to optimize weights in autoencoders [55]. Among Bayesian classifiers, a variant of naive Bayes that utilizes confidence intervals associated with each conditional probability in each predictor variable seeks the best combination of these parameters using the UMDA$_c^G$ algorithm [56]. Meanwhile, in semi-naive Bayes models, the UMDA algorithm has been applied to identify the model with the best performance [57]. In logistic regression, UMDA$_c^G$ has been used to search for parameter estimates that solve the system of $n + 1$ equations and $n + 1$ unknowns arising from the maximization of the conditional likelihood function [58].

The three most common types of **clustering**, namely hierarchical, partitional, and probabilistic, have been addressed using EDAs. In hierarchical clustering, UMDA has been applied to provide stochasticity to the subset merging process [59]. In partitional clustering, MIMIC, COMIT, and EBNA have been EDAs applied to the K-means algorithm [60], while UMDA and EBNA enhanced the search process for optimal exemplars in the affinity propagation algorithm [61]. Probabilistic clustering is based on the structural EM algorithm to search for probabilistic dependency relationships among variables. In [62], UMDA is incorporated to improve the greedy heuristic on which the structural EM typically relies.

In **Bayesian networks**, EDAs have been applied to optimization problems arising both in inference tasks and in learning the model from data. For inference, [63] uses a recursive EDA to search for the optimal triangulation of the moral graph associated with the Bayesian network, while [64] applies both UMDA and EBNA in counterfactual reasoning from Bayesian classifiers. Learning the structure and parameters of the Bayesian network in the space of DAGs has been carried out using UMDA and PBIL [65], while in the total ordering space, [66] has applied both discrete and continuous versions of UMDA and MIMIC.

Conclusions

EDAs are a type of evolutionary computation algorithm based on probability theory, enabling the evolution of a population of individuals through the learning of a probabilistic model that fits the selected individuals in each generation, followed by the simulation of this model to obtain the next generation of individuals. EDAs have been successfully applied to a wide range of real-world combinatorial optimization problems as well as in continuous domains, and have also been utilized for providing the best model in various machine learning tasks.

In this chapter, the focus has been on EDAs where probabilistic (directed) graphical models like (Gaussian) Bayesian networks are learned from the selected individuals. Other probabilistic models that have supported EDA methodologies include (undirected) Markov networks [67].

EDAs can play a significant role in interpreting the process followed by the metaheuristic in reaching the optimal solution. In this regard, analyzing the evolution of (Gaussian) Bayesian network structures learned across generations, along with landscape analysis via search trajectory networks [68], can be highly beneficial.

Cross-References

Local Search. Particle Swarm Optimization. Genetic Algorithms. Variable Neighborhood Search. Simulated Annealing. Ant Colony Optimization. Scatter Search. Tabu Search. Multimodal Optimization. Multiobjective Optimization.

References

1. P. Larrañaga, and J.A. Lozano (eds.) (2002). *Estimation of Distribution Algorithms. A New Tool for Evolutionary Computation*. Kluwer Academic Publishers.
2. M. Pelikan, D.E. Goldberg, and F. Lobo (2002). A survey of optimization by building and using probabilistic models. *Computational Optimization and Applications*, 21 (1), 5–20.
3. J. A. Lozano, P. Larrañaga, I. Inza, and E. Bengoetxea (eds.) (2005). *Towards an New Evolutionary Computation. Advances in Estimation of Distribution Algorithms*. Springer.
4. M. Hauschild, and M. Pelikan (2011). An introduction and survey of estimation of distribution algorithms. *Swarm and Evolutionary Computation*, 1 (3), 111–128.
5. R. Santana, J.A. Lozano, and P. Larrañaga (2008). Research topics in discrete estimation of distribution algorithms. *Memetic Computing*, 1, 35–54.
6. P. Larrañaga, H. Karshenas, C. Bielza, and R. Santana (2012). A review on probabilistic graphical models in evolutionary computation. *Journal of Heuristics*, 18 (5), 795–819.
7. G. Rudolph (1997). *Convergence Properties of Evolutionary Algorithms*. Kovač.
8. J. Ceberio, A. Mendiburu, and J. A. Lozano (2022). A roadmap for solving optimization problems with estimation of distribution algorithms. *Natural Computing*, 23 (1), 99–113.
9. B.-T. Zhang, and J.-J. Kim (2000). Comparison of selection methods for evolutionary optimization. *Evolutionary Optimization*, 2 (1), 55–70.
10. P. Larrañaga, R. Etxeberria, J.A. Lozano, and J.M. Peña (2000). Optimization in continuous domains by learning and simulation of Gaussian networks. *Proceedings of the 2000 Genetic and Evolutionary Computation Conference Workshop Program*, 201–204, ACM.
11. H. Mühlenbein, and G. Paaß (1996). From recombination of genes to the estimation of distributions. I. Binary parameters. *Lecture Notes in Computer Science*, vol. 1411, 178–187, Springer.

12. S. Baluja (1994). *Population-Based Incremental Learning: A Method for Integrating Genetic Search Based Function Optimization and Competitive Learning.* Technical Report CMU-CS-94-163, Carnegie Mellon University.

13. M. Sebag, and A. Ducoulombier (1998). Extending population-based incremental learning to continuous spaces. *Parallel Problem Solving from Nature- PPSN V*, 418–427, Springer.

14. G. Harik, F.G. Lobo, and D.E. Goldberg (1999). The compact genetic algorithm. *IEEE Transactions on Evolutionary Computation*, 3 (4), 287–297.

15. B. Doerr, and M. Dufay (2022). General univariate estimation-of-distribution algorithms. *Proceedings of the 17th International Conference on Parallel Problem Solving from Nature*, 470–484, Springer.

16. J.S. De Bonet, C.L. Isbell, and P. Viola (1997). MIMIC: Finding optima by estimating probability densities. *Advances in Neural Information Processing Systems*, 9, 424–430.

17. S. Baluja, and S. Davies (1997). *Combining Multiple Optimization Runs with Optimal Dependency Trees.* Technical Report CMU-CS-97-157, Carnegie Mellon University.

18. R. Salinas-Gutiérrez, A. Hernández-Aguirre, and E. R. Villa-Diharce (2011). Dependence trees with copula selection for continuous estimation of distribution algorithms. *Proceedings of the 13th Annual Conference on Genetic and Evolutionary Computation*, 585–592, ACM.

19. M. Pelikan, and H. Mühlenbein (1999). The bivariate marginal distribution algorithm. *Advances in Soft Computing-Engineering Design and Manufacturing*, 521–535, Springer.

20. Z. Lin, Q. Su, and G. Xie (2021). NMIEDA: Estimation of distribution algorithm based on normalized mutual information. *Concurrency and Computation: Practice and Experience*, 33 (6), e6074.

21. D. Koller, and N. Friedman (2009). *Probabilistic Graphical Models: Principles and Techniques.* The MIT Press.

22. R. Etxeberria, and P. Larrañaga (1999). Global optimization using Bayesian networks. *Second International Symposium on Artificial Intelligence*, 332–339.

23. P. Larrañaga, R. Etxeberria, J.A. Lozano, and J.M. Peña (2000). Combinatorial optimization by learning and simulation of Bayesian networks. *Proceedings of the 16th Conference on Uncertainty in Artificial Intelligence*, 343–352, Morgan Kaufmann Publishers.

24. W. Dong, T. Chen, P. Tiňo, and X. Yao (2013). Scaling up estimation of distribution algorithms for continuous optimization. *IEEE Transactions on Evolutionary Computation*, 17 (6), 797–822.

25. M. Pelikan, D.E. Goldberg, and E. Cantú-Paz (1999). BOA: The Bayesian optimization algorithm. *Proceedings of the 1st Annual Conference on Genetic and Evolutionary Computation*, vol. 1, 525–532.

26. H. Mühlenbein, and T. Mahning (1999). FDA - A scalable evolutionary algorithm for the optimization of additively decomposed functions. *Evolutionary Computation*, 7(4), 353–376.

27. H. Xu, J. Yang, P. Jia, and Y. Ding (2013). Effective structure learning for estimation of distribution algorithms via L1-regularized Bayesian networks. *International Journal of Advanced Robotic Systems*, 10 (1).

28. H. Karshenas, R. Santana, C. Bielza, and P. Larrañaga (2013). Regularized continuous estimation of distribution algorithms. *Applied Soft Computing*, 13(5), 2412–2432.

29. G. Harik (1999). *Linkage Learning via Probabilistic Modelling in the ECGA*, IlliGAL Report No. 99010, University of Illinois at Urbana-Champaign.

30. M. Probst, and F. Rothlauf (2020). Harmless overfitting: Using denoising autoencoders in estimation of distribution algorithms. *Journal of Machine Learning Research*, 21 (78), 1–31.

31. L. Bao, X. Sun, D. Gong, Y. Zhang, and B. Xu (2020). Enhanced interactive estimation of distribution algorithms with attention mechanism and restricted Boltzmann machine. *2020 IEEE Congress on Evolutionary Computation*, 1–9.

32. V. Soloviev, C. Bielza, P. Larrañaga (2024). Semiparametric estimation of distribution algorithms for continuous optimization. *IEEE Transactions on Evolutionary Computation*, 28(4), 1069–1083.

33. J. Ceberio, E. Irurozki, A. Mendiburu, and J.A. Lozano (2012). A review on estimation of distribution algorithms in permutation-based combinatorial optimization problems. *Progress in Artificial Intelligence*, 1, 103–117.

34. M. Pelikan, and D.E. Goldberg (2000). Genetic algorithms, clustering, and the breaking of symmetry. *Proceedings of Parallel Problem Solving from Nature IV*, 385–394, Springer.

35. P.A.N. Bosman, and D. Thierens (2002). Multi-objective optimization with diversity preserving mixture-based iterated density estimation evolutionary algorithms. *International Journal of Approximate Reasoning*, 31(3), 259–289.

36. J.M. Peña, J.A. Lozano, and P. Larrañaga (2005). Globally multimodal problem optimization via an estimation of distribution algorithm based on unsupervised learning of Bayesian networks. *Evolutionary Computation*, 13 (1), 43–66.

37. M. Pelikan, K. Sastry, and D.E. Goldberg (2006). Multiobjective estimation of distribution algorithms. In *Scalable Optimization via Probabilistic Modeling*, vol. 33, 223–248, Springer.

38. H. Karshenas, R. Santana, C. Bielza, and P. Larrañaga (2013). Multi-objective estimation of distribution algorithms based on joint modelling of objectives and variables. *IEEE Transactions on Evolutionary Computation*, 18(4), 519–542.

39. G. Uludağ, B. Kiraz, A.S.E. Uyar, and E. Özcan (2013). A hybrid multi-population framework for dynamic environments combining online and offline learning. *Soft Computing*, 17 (12), 2327–2348.

40. M. Jiang, Z. Huang, L. Qiu, W. Huang, and G. G. Yen (2018). Transfer learning-based dynamic multiobjective optimization algorithms. *IEEE Transactions on Evolutionary Computation*, 22 (4), 501–514.

41. J.A. Lozano. R. Sagarna, and P. Larrañaga (2002). Parallel estimation of distribution algorithms. In *Estimation of Distribution Algorithms. A New Tool for Evolutionary Computation*, 129–145, Kluwer Academic Publishers.

42. A. Mendiburu, J.A. Lozano, and J. Miguel-Alonso (2005). Parallel implementation of EDAs based on probabilistic graphical models. *IEEE Transactions on Evolutionary Computation*, 9 (4), 406–423.

43. A. Zhou, J. Sun, and Q. Zhang (2005). An estimation of distribution algorithm with cheap and expensive local search methods. *IEEE Transactions on Evolutionary Computation*, 19 (6), 807–822.

44. J. Sun, Q. Zhang, and E.P.K. Tsang (2005). DE/EDA: A new evolutionary algorithm for global optimization. *Information Sciences*, 169 (3-4), 249–262.

45. R. Blanco, I. Inza, M. Merino, J. Quiroga, and P. Larrañaga (2005). Feature selection in Bayesian classifiers for the prognosis of survival of cirrhotic patients treated with TIPS. *Journal of Biomedical Informatics*, 38, 376–388.

46. R. Santana, J.A. Lozano, and P. Larrañaga (2008). Protein folding in simplified models with estimation of distribution algorithms. *IEEE Transactions on Evolutionary Computation*, 12(4), 418–438.

47. S. Mishra, R.S. Modak, and S. Ganesan (2010). Optimization of depleted uranium bundle loading in fresh core of Indian PHWR by evolutionary algorithm. *Annals of Nuclear Energy*, 37 (2), 208–217.

48. Y. Liang, Z. Ren, L. Wang, H. Liu, and W. Du (2021). Surrogate-assisted cooperative signal optimization for large-scale traffic networks. *Knowledge-Based Systems*, 234, 107542.

49. Z. Xu, Y. Wang, S. Li, Y. Liu, Y. Todo, and S. Gao (2016). Immune algorithm combined with estimation of distribution for traveling salesman problem. *IEEJ Transactions on Electrical and Electronic Engineering*, 11 (S1), 142–154.

50. M.S.R. Martins, M. El Yafrani, M.R.B.S. Delgado, M. Wagner, B. Ahiod, and R. Lüders (2017). HSEDA: A heuristic selection approach based on estimation of distribution algorithm for the travelling thief problem. *Proceedings of the 2017 Genetic and Evolutionary Conference*, 361–368, ACM.

51. P. Larrañaga, and C. Bielza (2024). Estimation of distribution algorithms in machine learning: A survey. *IEEE Transactions on Evolutionary Computation*, 28(5), 1301–1321.

52. I. Inza, P. Larrañaga, R. Etxeberria, and B. Sierra (2000). Feature subset selection by Bayesian network–based optimization. *Artificial Intelligence*, 123, 157–184.

53. V. Robles, C. Bielza, P. Larrañaga, S. González, and L. Ohno-Machado (2008). Optimizing logistic regression coefficients for discrimination and calibration using estimation of distribution algorithms. *TOP*, 16(2), 345–366.

54. S. Baluja (1995). *An Empirical Comparison of Seven Iterative and Evolutionary Function Optimization Heuristics*, Technical Report CMU-CS-95-193, Carnegie Mellon University.

55. Q. Xu, A. Liu, X. Yuan, Y. Song, C. Zhang, and Y. Li (2021). Random mask-based estimation of the distribution algorithm for stacked auto-encoder one-step pre-training. *Computers and Industrial Engineering*, 158, 107400.

56. V. Robles, P. Larrañaga, J.M. Peña, E. Menasalvas, and M. S. Pérez (2003). Interval estimation naïve Bayes. *Lecture Notes in Computer Science*, 2810, 143–154, Springer.

57. V. Robles, P. Larrañaga, J.M. Peña, E. Menasalvas, M.S. Pérez, V. Herves, and A. Wasilewska (2004). Bayesian networks multi-classifiers for protein secondary structure prediction. *Artificial Intelligence in Medicine*, 31, 117–136.

58. C. Bielza, V. Robles, and P. Larrañaga (2011). Regularized logistic regression without a penalty term: An application to cancer classification with microarray data. *Expert Systems with Applications*, 38(5), 5110–5118.

59. J. Fan (2019). OPE-HCA: An optimal probabilistic estimation approach for hierarchical clustering algorithm. *Neural Computing and Applications*, 31, 2095–2105.

60. J. Roure, P. Larrañaga, and R. Sangüesa (2002). An empirical comparison between K-means, GAs and EDAs in partitional clustering. In *Estimation of Distribution Algorithms. A New Tool*

for Evolutionary Computation, Kluwer Academic Publishers, 343–360.

61. R. Santana, C. Bielza, and P. Larrañaga (2011). Affinity propagation enhanced by estimation of distribution algorithms. *Proceedings of the 2011 Genetic and Evolutionary Conference*, 331–338, ACM.
62. J.M. Peña, J.A. Lozano, and P. Larrañaga (2004). Unsupervised learning of Bayesian networks via estimation of distribution algorithms: An application to gene expression data clustering. *International Journal of Uncertainty, Fuzziness and Knowledge-Based Systems*, 12, 63–82.
63. T. Romero, and P. Larrañaga (2009). Triangulation of Bayesian networks with recursive estimation of distribution algorithms. *International Journal of Approximate Reasoning*, 50(3), 472–484.
64. D. Zaragoza, C. Bielza, and P. Larrañaga (2024). Multi-objective counterfactuals in Bayesian classifiers with estimation of distribution algorithms. *Proceedings of the 12th International Conference on Probabilistic Graphical Models*, PMLR 246, 415–426.
65. R. Blanco, I. Inza, and P. Larrañaga (2003). Learning Bayesian networks in the space of structures by estimation of distribution algorithms. *International Journal of Intelligent Systems*, 18, 205–220.
66. T. Romero, P. Larrañaga, and B. Sierra (2004). Learning Bayesian networks in the space of orderings with estimation of distribution algorithms. *International Journal of Pattern Recognition and Artificial Intelligence*, 18 (4), 607–625.
67. S. Shakya, and R. Santana (eds.) (2012). *Markov Networks in Evolutionary Computation*. Springer.
68. M. Fyvie, J.A.W. McCall, L.A. Christie, A.E.I. Brownlee, and M. Singh (2023). Towards explainable metaheuristics: Feature extraction from trajectory mining. *Expert Systems*, e13494.